

Guión del proyecto final (Parte 2)

Metodología de la Programación. Curso 2025/26

Grado en Ingeniería Informática. Grupo F

1 Sobre esta práctica

El objetivo de la práctica final es construir una plataforma de streaming de música (llamada "Spotifry") en el que distintos usuarios podrán guardar y reproducir canciones, crear listas de reproducción, entre otras funcionalidades.

En esta segunda parte, se desarrollará la parte de la aplicación que se ocupará de la carga de datos desde ficheros para que los datos de la aplicación sean persistentes y la creación de un **dashboard** en HTML con información personalizada de los usuarios.

Se recuerda que la realización de la práctica final es INDIVIDUAL. Cualquier copia detectada supondrá una calificación de 0 en dicha práctica para todos los estudiantes implicados en la copia. Además, queda expresamente prohibido el uso de herramientas de Inteligencia Artificial generativa para la resolución de la práctica. La autoría de la práctica será verificada mediante entrevistas individuales y/o pruebas prácticas, la no superación de dicha defensa conlleva una calificación de 0 en dicha práctica

Para el desarrollo de la práctica es OBLIGATORIO utilizar compilación separada (con *makefile*), de forma que para cada clase se deberá separar la definición de la misma en un fichero `.h` (o `.h/.hpp`) y los detalles de su implementación en otro fichero `.cpp` (o `.cc`). Recordad que los nombres de los ficheros de las clases deben (en minúscula) coincidir con los nombres de sus clases.

Se valorará una adecuada modularización del código, evitando en la medida de lo posible la duplicación de código innecesario. Además, se valorará la limpieza del código, utilizando nombres de variables y métodos descriptivos y una adecuada indentación del código.

Junto con el enunciado de la práctica se proporciona un ejemplo de programa principal para probar los distintos métodos de las clases. No obstante, la práctica podrá ser verificada con otros programas principales adicionales al proporcionado.

Para la entrega de la práctica final se deberán proporcionar todos los ficheros fuente del proyecto, así como un fichero *makefile* que permita la compilación automática del proyecto en un entorno GNU/Linux.

Recomendaciones:

1. **Lee detenidamente todo este guion antes de programar nada.**
2. Realiza la segunda parte de la práctica como un proyecto independiente de la primera. Empieza a trabajar en la segunda parte con una copia del proyecto de la parte 1.

3. **Sigue un desarrollo incremental**, añadiendo funcionalidades de forma progresiva, una vez se han probado las que se han añadido anteriormente.
4. Desarrolla todos aquellos métodos privados que consideres necesarios para permitir una adecuada modularización del programa, evitando en la medida de lo posible repetir el mismo código en distintas partes del mismo.

2 Gestión de Referencias Circulares y Compilación

En proyectos de C++ con múltiples clases interrelacionadas es común encontrarse con el problema de las **referencias circulares**. Esto ocurre cuando la Clase A necesita conocer a la Clase B, y la Clase B a la Clase A.

2.1 Solución: Forward Declaration (Declaración Adelantada)

La solución consiste en desacoplar las dependencias en los archivos de cabecera:

1. **En el archivo .h:** Si solo vas a declarar un puntero o una referencia a otra clase, no uses `#include`. En su lugar, usa una **declaración adelantada**: `class NombreDeLaClase;`. Esto indica al compilador que ese tipo existe sin necesidad de leer todo su archivo.
2. **En el archivo .cpp:** Es aquí donde debes incluir el `.h` real de la clase que necesitas para poder acceder a sus métodos y atributos.

3 Tutorial de Gestión de Ficheros (Persistencia)

La persistencia de datos en C++ se gestiona mediante la librería `<fstream>`. El flujo de trabajo estándar sigue siempre una estructura de tres pasos: **Apertura, Procesamiento y Cierre**.

3.1 Conceptos Generales

Para operar con ficheros, utilizamos dos tipos de flujos principales:

- **ifstream:** Para la lectura de datos (Input File Stream).
- **ofstream:** Para la escritura de datos (Output File Stream).

3.2 Ciclo de Vida de un Fichero

El ciclo de gestión puede resumirse en el siguiente esquema formal:

$$\text{Ciclo} = \begin{cases} 1. & \text{Apertura: } \text{archivo.open("ruta.txt");} \\ 2. & \text{Validación: } \text{if (!archivo.is_open()) \{ error \}} \\ 3. & \text{Lectura: } \text{while (getline(archivo, linea)) \{ ... \}} \\ 4. & \text{Cierre: } \text{archivo.close();} \end{cases}$$

3.3 Técnicas de Lectura y Parsing

Existen dos formas de extraer información:

- **Operador de extracción (»):** Lee palabra por palabra, deteniéndose en espacios o saltos de línea. Es ideal para números o identificadores simples.
- **getline():** Lee una línea completa o hasta un delimitador específico (como ';' o '|'). Es la herramienta clave para procesar formatos tipo CSV como el de esta práctica.

3.4 Resumen de Buenas Prácticas

- Siempre compruebe si el fichero se ha abierto correctamente antes de leer.
- Use `close()` explícitamente para liberar el recurso lo antes posible.
- Al leer listas separadas por cualquier separador, verifique si el token está vacío para evitar procesar elementos inexistentes.

4 Ficheros

Se proporcionan cuatro ficheros CSV (separador: punto y coma) con los datos de los usuarios, las canciones, los álbumes y las playlist y canciones favoritas de cada usuario. Además se proporciona el fichero HTML básico para la parte de generación del dashboard. A continuación se describe el formato de cada uno:

- **Fichero de usuarios (users.txt):** cada fila proporciona la información de un usuario con el formato "name;email;birthdate;gender".

Ejemplo:

```
Alice Johnson;alice.j@mail.com;1990/05/12;FEMALE
```

- **Fichero de canciones (songs.txt):** cada fila contiene la información de una canción con el formato "song name;genre;duration".

Ejemplo:

```
One More Time;ELECTRONIC;320;Discovery
```

- **Fichero de álbumes (albums.txt):** cada álbum ocupa una fila con el formato "title;release date;artist;num_songs;[canciones...]".

Ejemplo:

```
Discovery;2001/03/12;Daft Punk;4;One More Time;Aerodynamic;Digital Love;  
Harder Better Faster Stronger
```

- Además, se proporciona un **fichero de persistencia**, en formato CSV, que permite guardar las canciones y las playlist favoritas de todos los usuarios (`spotify_db.txt`). Este fichero contiene dos tipos de líneas:

```
1) USER;email:[songTitle1|songTitle2|...]
2) PLAYLIST;title;creator_email;creation_date;privacy:[songTitle1|...];[followerEmail1|...]
```

Por ejemplo:

```
USER;alice.j@mail.com;Thriller|Saoko|Time
PLAYLIST;Rock Classics;bob.smith@provider.net;2024/05/10;PUBLIC;Back in Black|Hells Bells|
Shoot to Thrill|Thunderstruck|Bohemian Rhapsody;alice.j@mail.com|charlie.b@music.org
```

El fichero de persistencia utiliza un sistema de etiquetas de cabecera para distinguir entre los dos tipos de registros que componen el estado del sistema. Las líneas que comienzan con el identificador **USER** contienen la información de perfil del usuario seguida de una lista de títulos de sus canciones favoritas. Por otro lado, las líneas identificadas con la cabecera **PLAYLIST** almacenan los metadatos de las listas creadas, incluyendo el correo electrónico del propietario, el estado de privacidad y dos sublistas críticas: una con los títulos de las canciones que la integran y otra con los correos de los usuarios seguidores. En ambos casos, mientras que el punto y coma (;) actúa como separador de campos principales, se emplea la barra vertical (|) para delimitar los elementos dentro de las sublistas, permitiendo así un parsing eficiente y sin ambigüedades.

- Para la realización de la parte del **dashboard** se proporciona un fichero `template.txt` con el contenido básico de la página del usuario en formato HTML. El contenido de este fichero se tratará como un `string` para la generación del `dashboard`.

5 Clases Lista de Álbumes y Artistas

Para almacenar el conjunto de las álbumes de nuestra aplicación Spotify habrá que crear una clase que se encargue de leer de fichero la información de los álbumes.

La definición de estas clases será la siguiente:

```
1      const int MAX_ARTISTS = 1000;
2
3      class ArtistList {
4          private:
5              int current_size;
6              Artist* list[MAX_ARTISTS];
7      };

```

```
1      const int MAX_ALBUMS = 1000;
2

```

```
3     class AlbumList {  
4         private:  
5             int current_size;  
6             Album* list[MAX_USERS];  
7     };
```

5.1 Constructores

Implementar los correspondientes constructores con parámetros que inicializará los distintos atributos de las clases. Cada uno recibirá la ruta de fichero desde el que se cargarán los datos (en ambos casos, el mismo albums.txt).

```
ArtistList(string file_path);
```

```
AlbumList(string file_path, ArtistList &artistList, SongList &songList);
```

A la hora de implementar el constructor se valorará una adecuada modularización del mismo y el control de los errores que se puedan producir.

5.2 Métodos get y set

Implementar todos los métodos get y set que consideres necesarios.

5.3 Método para escribir en fichero (AlbumList)

Este método será el encargado de recorrer la lista y almacenar en un fichero todos sus elementos, siguiendo el formato descrito anteriormente para el fichero album.txt. La ruta del fichero se pasará como parámetro.

```
bool dump(string file_path);
```

5.4 Método para mostrar por pantalla los datos de un álbum (AlbumList)

```
void printAlbum(string title) const;
```

Deberá mostrar por pantalla la información de un album de la siguiente forma:

Album: Nevermind, Artist: Nirvana, Release Date: 1991

Songs:

- Smells Like Teen Spirit Genre: ROCK, Duration: 3m 28s
- Come as You Are Genre: ROCK, Duration: 3m 15s
- Lithium Genre: ROCK, Duration: 5m 25s
- Breed Genre: ROCK, Duration: 3m 30s
- On a plane Genre: ROCK, Duration: 3m 0s

5.5 Destruectores

Implementar los destructores cuando sea necesario.

6 Clase Lista de Canciones

Para almacenar el conjunto de las canciones de nuestra aplicación Spotify habrá que crear una clase que se encargue de leer de fichero las canciones y las almacene.

La definición de esta clase será la siguiente:

```
1  const int BLOCK_MAX_SIZE = 10;
2
3  struct Block {
4      Song* list[BLOCK_MAX_SIZE];
5      int current_size;
6      Block* next;
7  };
8
9  class SongList {
10     private:
11         Block* firstBlock;
12         Block* lastBlock;
13 };
```

6.1 Constructor de la clase SongList

Implementar un constructor con parámetros que inicializará los distintos atributos de la clase SongList. Este constructor recibirá como parámetro una ruta de fichero desde el que se cargarán los datos de las canciones y se encargará de reservar la memoria necesaria para almacenar los datos del fichero. El tamaño de cada bloque será de BLOCK_MAX_SIZE canciones, el atributo current_size nos indicará el número de posiciones ocupadas en ese bloque y el atributo next es un puntero al siguiente bloque. Tendremos un puntero al primer bloque de juegos (firstBlock) y un puntero al último bloque (lastBlock). El atributo lastBlock.next debe apuntar siempre a NULL (esto es, el siguiente bloque al que apunta el último bloque debe ser NULL, ya que estamos en el último).

```
SongList(string file_path);
```

A la hora de implementar el constructor se valorará una adecuada modularización del mismo y el control de los errores que se puedan producir.

6.2 Métodos get y set

Implementar todos los métodos get y set que consideres necesarios.

6.3 Método para añadir canción

Implementar un método que añada una nueva canción a la colección, que se añadirá en el último bloque, siempre que haya espacio suficiente. De no ser así, se deberá reservar memoria para un nuevo bloque. La canción solo se añadirá si existe el album correspondiente. En caso afirmativo, se añadirá la canción al album.

```
bool addSong(string title, Genre genre, int duration, AlbumList &albumList, string  
    ↪ album);
```

6.4 Método para buscar canciones

Implementar un método para buscar canciones a partir de una cadena de texto. Busca canciones que contengan **searchTerm** en el título, sean del género **genre** y su duracion esté comprendida entre **durationMin** y **durationMax**. Además:

- Si **searchTerm** es vacío se ignora este criterio.
- Si **genre** es UNKNOWN se ignora este criterio.
- Si **durationMin** o **durationMax** son -1 se ignoran uno o ambos.

```
bool SongList::findSong(string searchTerm, Genre genre, int durationMin, int  
    ↪ durationMax, Song ** results, int maxResults, int &numResults) {
```

Los resultados se devolverán en el array **results** (un vector dinámico, cuya liberación es responsabilidad del usuario de este método) y el tamaño de ese array se devolverá en **numResults**. Como máximo se devolverán **maxResults**, que será la memoria reservada.

6.5 Destructor de la clase SongList

Implementar el destructor de la clase SongList.

7 Clase UserList

```
1  const int MAX_USERS=1000;
2  const int MAX_PLAYLISTS=2000;
3
4  class UserList {
5      private:
6          int current_size;
7          User* list[MAX_USERS];
8
9          int number_of_playlists;
10         Playlist* playlists[MAX_PLAYLISTS];
11     }
```

7.1 Constructor de la clase UserList

Implementar el constructor, que recibirá la ruta del fichero de usuarios.

```
UserList(string users_file_path, string spotify_db_file_path);
```

A partir de la información de los dos ficheros, se creará la lista de usuarios de la aplicación y la lista de playlists. Se valorará una adecuada modularización del constructor y una correcta gestión de los errores.

7.2 Añadir usuario o lista de reproducción

Implementar métodos para añadir un nuevo usuario o una nueva lista de reproducción.

```
bool addUser(User* user_to_add);
bool addPlaylist(Playlist* playlist_to_add);
```

Para que un usuario se pueda añadir correctamente, no debe existir previamente en la lista. Tampoco se puede añadir una playlist que ya exista en la lista, y tampoco se puede añadir una playlist si el usuario que la ha creado no está en la lista de usuarios. Si la lista está llena, no se podrá añadir nada más.

7.3 Métodos get y set

Implementar todos los métodos get y set que considere necesarios.

7.4 Destructor UserList

Implementar el destructor de la clase UserList.

8 Clase Spotifry

```
1 class Spotifry {  
2     private:  
3         SongList songs;  
4         AlbumList albums;  
5         ArtistList artists;  
6         UserList users;  
7 }
```

8.1 Método para crear una nueva lista de reproducción

Implementar un método para crear una nueva lista de reproducción y añadirla a la lista interna.

```
bool createPlaylist(string user_email, string playlist_title,  
                    int num_songs, string *song_names);
```

Sólo se podrá crear la playlist si el usuario con el email dado existe en Spotifry, y si todas las canciones cuyos nombres se pasan como argumento (exttttsong_names) existen literalmente en Spotifry.

9 Programa Principal

Junto con este guion se proporciona un ejemplo de programa principal para probar la funcionalidad de las distintas clases desarrolladas en el guion. El software proporcionado debe funcionar con este programa principal sin realizar ningún cambio. Esto no implica que no se deban de realizar pruebas adicionales para comprobar el correcto funcionamiento del programa.

10 Dashboard Usuario

En este parte desarrollaremos una aplicación en C++ capaz de generar automáticamente un dashboard musical inspirado en plataformas modernas como Spotify. El objetivo es transformar los datos almacenados por el sistema de información musical en una interfaz visual atractiva y dinámica utilizando HTML y CSS.

El dashboard generado mostrará:

- Información del usuario
- Estadísticas musicales
- Playlists
- Canciones favoritas

- Un reproductor visual estilo Spotify

La aplicación actuará como un pequeño motor de plantillas:

1. Leerá una plantilla HTML desde fichero.
2. Sustituirá etiquetas del tipo `{{TAG}}`.
3. Generará automáticamente el contenido visual.
4. Exportará el resultado final a una página web HTML.

10.1 Funciones a desarrollar

Todos los métodos detallados a continuación se desarrollarán dentro de la clase `Spotifry`.

10.1.1 Lectura de plantilla

```
string leerArchivo(const string &nombre);
```

Debe leer el contenido del fichero HTML y devolverlo como un string.

```
string leerArchivo(const string &nombre)
{
    ifstream f(nombre);

    stringstream buffer;

    buffer << f.rdbuf();

    return buffer.str();
}
```

10.1.2 Sustitución de etiquetas

```
string reemplazar(string html, const string &tag, const string &valor);
```

Debe sustituir todas las apariciones de una etiqueta por su valor correspondiente.

Generación de playlists

```
string generarPlaylists(const Usuario &u, string &html);
```

La plantilla HTML contiene la siguiente etiqueta:

```
{{PLAYLISTS}}
```

Esta etiqueta deberá sustituirse por varios bloques HTML generados automáticamente desde C++.

Cada playlist del usuario deberá convertirse en un bloque HTML similar al siguiente:

```
<div class='playlist'>

<div class='play-title'>
Road Trip
</div>

<div class='play-info'>
15 canciones
</div>

<div class='play-info'>
58 min
</div>

</div>
```

El alumno deberá recorrer el vector de playlists del usuario y concatenar todos los bloques HTML en un único string.

10.1.3 Generación de canciones

```
string generarCanciones(const Usuario &u);
```

Debe generar las filas HTML correspondientes a las canciones favoritas.

```
<tr>
<td>
<img src='img/music.png'
      width='40'
      height='40'>
</td>
```

```
<td>Viva la Vida</td>  
<td>Coldplay</td>  
<td>4:03</td>  
  
</tr>
```

Si el usuario tiene varias canciones favoritas, la etiqueta:

```
{{CANCIONES}}
```

deberá sustituirse automáticamente por varias filas HTML dentro de la tabla de canciones.

10.1.4 Resultado Final

El resultado obtenido será un fichero HTML con la siguiente apariencia:

 **Music Dashboard**
Perfil musical de Alice Johnson

Resumen

4

Playlists

8

Favoritas

Coldplay

Top artista

Playlists

Rock Classics

4 canciones

Daft Punk Essentials

4 canciones

Bowie Greatest

4 canciones

Daft Punk Essentials

4 canciones

Canciones favoritas

	TÍTULO	ARTISTA	DURACIÓN
	Thriller	Michael Jackson	5:14
	Sacko	Rosalía	4:01
	Time	Pink Floyd	7:46
	One More Time	Daft Punk	3:31
	Digital Love	Daft Punk	6:39
	Starman	David Bowie	7:49
	Billie Jean	Michael Jackson	4:09
	Hells Bells	AC/DC	7:57

► Viva la Vida - Coldplay

1:32 / 4:03